

✓ SESSION 8 — Useful Utility Functions in NumPy

Utility functions in NumPy help perform common operations required in data processing and analysis.

These functions are widely used for:

- Data cleaning
- Sorting data
- Identifying missing values
- Checking logical conditions
- Handling special values such as NaN and infinity

In this session we will study:

1. np.unique()
2. np.sort() and np.argsort()
3. np.where()
4. np.nonzero()
5. np.any() and np.all()
6. np.isnan(), np.isinf(), np.isfinite()
7. np.clip()
8. Handling NaN values

1. np.unique()

Purpose

Returns the unique elements of an array by removing duplicate values.

Syntax

np.unique(array)

Parameters

array : input array

Returns

Sorted array of unique values

```
import numpy as np

arr = np.array([1, 2, 2, 3, 3, 4, 5, 5])

unique_values = np.unique(arr)
print(np.unique(arr))

print("Original Array:", arr)
print("Unique Values:", unique_values)

[1 2 3 4 5]
Original Array: [1 2 2 3 3 4 5 5]
Unique Values: [1 2 3 4 5]
```

✓ 2. Sorting Functions

Sorting is important in data preprocessing and statistical analysis.

NumPy provides two useful functions:

- np.sort()
 - np.argsort()
-

np.sort()

Returns a sorted copy of the array.

Syntax

np.sort(array)

Parameters

array : input array

Returns

Sorted array

```
arr = np.array([30, 10, 50, 20])

sorted_arr = np.sort(arr)

print("Original Array:", arr)
print("Sorted Array:", sorted_arr)
```

```
Original Array: [30 10 50 20]
Sorted Array: [10 20 30 50]
```

np.argsort()

Instead of returning sorted values, this function returns the indices that would sort the array.

Syntax

np.argsort(array)

Returns

Indices of elements in sorted order

```
arr = np.array([30, 10, 50, 20])

indices = np.argsort(arr)
print(indices)
print("Array:", arr)
print("Indices that sort the array:", indices)

print("Sorted using indices:", arr[indices])
```

```
[1 3 0 2]
Array: [30 10 50 20]
Indices that sort the array: [1 3 0 2]
Sorted using indices: [10 20 30 50]
```

```
arr=np.array([90,99,28,386,273,929,2,1929,38,200,283,48,398,29,99,99])
print(np.argsort(arr))
arr=np.unique(arr)
print(arr)
print(np.argsort(arr))
```

```
[ 6  2 13  8 11  0 14  1 15  9  4 10  3 12  5  7]
[  2  28  29  38  48  90  99 200 273 283 386 398 929 1929]
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13]
```

```
A = {1,2,567,3,1,2,1,4,5}
print(A)
```

```
{1, 2, 3, 4, 5, 567}
```

```
# a = {7,3,28,9,9,10,28,20}
a = {7,3,28,9,9,10,30}
print(a)
```

```
{3, 7, 9, 10, 28, 30}
```

3. np.where()

np.where() is used to find indices where a condition is true.

It is very useful for conditional selection and filtering.

Syntax

np.where(condition)

Returns

Indices where the condition is satisfied

```
arr = np.array([5, 10, 15, 20, 25])

indices = np.where(arr > 15)

print("Array:", arr)
print("Indices where values > 15:", indices)
print("Values greater than 15:", arr[indices])
```

```
Array: [ 5 10 15 20 25]
Indices where values > 15: (array([3, 4]),)
Values greater than 15: [20 25]
```

4. np.nonzero()

This function returns the indices of non-zero elements.

Syntax

np.nonzero(array)

Returns

Indices of elements that are not equal to zero.

```
arr = np.array([0, 1, 0, 2, 3, 0])

print("Array:", arr)

non_zero_indices = np.nonzero(arr)

print("Indices of non-zero elements:", non_zero_indices)
print("Non-zero values:", arr[non_zero_indices])
```

```
Array: [0 1 0 2 3 0]
Indices of non-zero elements: (array([1, 3, 4]),)
Non-zero values: [1 2 3]
```

5. np.any() and np.all()

These functions are used to check logical conditions on arrays.

np.any()

Returns True if at least one element satisfies the condition.

Syntax

np.any(condition)

np.all()

Returns True only if all elements satisfy the condition.

Syntax

`np.all(condition)`

```
arr = np.array([5, 10, 11])

print("Any element greater than 12:", np.any(arr > 12))
print("All elements greater than 2:", np.all(arr > 4))
```

```
Any element greater than 12: False
All elements greater than 2: True
```

6. Checking Special Values

Real-world datasets often contain special values such as:

- NaN (Not a Number)
- Infinity
- Negative Infinity

NumPy provides functions to identify them.

`np.isnan()`

Checks whether elements are NaN.

`np.isinf()`

Checks whether elements are infinite.

`np.isfinite()`

Checks whether elements are finite values.

```
arr = np.array([1, np.nan, np.inf, -np.inf, 5])

print("Array:", arr)

print("isnan:", np.isnan(arr))
print("isinf:", np.isinf(arr))
print("isfinite:", np.isfinite(arr))
```

```
Array: [ 1. nan inf -inf  5.]
isnan: [False  True False False False]
isinf: [False False  True  True False]
isfinite: [ True False False False  True]
```

7. `np.clip()`

`np.clip()` limits values within a specified range.

Values below the minimum are replaced with the minimum value. Values above the maximum are replaced with the maximum value.

Syntax

`np.clip(array, min_value, max_value)`

```
arr = np.array([2, 5, 10, 15, 20])

clipped = np.clip(arr, 8, 12)

print("Original Array:", arr)
print("Clipped Array:", clipped)
```

```
Original Array: [ 2  5 10 15 20]
Clipped Array: [ 8  8 10 12 12]
```

8. Handling NaN Values

NaN stands for "Not a Number".

It is used to represent missing or undefined values.

Example:

`np.nan`

Standard statistical functions return NaN if the dataset contains NaN values.

To handle this, NumPy provides special functions:

- `np.nanmean()`
- `np.nanstd()`
- `np.nansum()`

These functions ignore NaN values during computation.

```
arr = np.array([10, 20, np.nan, 40])

print("Array:", arr)

print("Mean (normal):", np.mean(arr))
print("Mean ignoring NaN:", np.nanmean(arr))

print("Standard deviation ignoring NaN:", np.nanstd(arr))

print("Sum ignoring NaN:", np.nansum(arr))
```

```
Array: [10. 20. nan 40.]
Mean (normal): nan
Mean ignoring NaN: 23.333333333333332
Standard deviation ignoring NaN: 12.472191289246473
Sum ignoring NaN: 70.0
```

Summary of Session 8

Functions covered:

Data inspection

- `np.unique()`

Sorting

- `np.sort()`
- `np.argsort()`

Conditional operations

- `np.where()`
- `np.nonzero()`

Logical checks

- `np.any()`
- `np.all()`

Special value detection

- `np.isnan()`
- `np.isinf()`
- `np.isfinite()`

Value limiting

- `np.clip()`

Handling missing values

- `np.nan`
- `np.nanmean()`

- `np.isnanstd()`
- `np.nansum()`

These functions are extremely important for:

- Data cleaning
- Preprocessing datasets
- Machine learning pipelines
- Real-world data analysis

Start coding or [generate](#) with AI.